



TÓPICOS EM INFORMÁTICA

*Textura, sprite batch e otimização,
blending, texture region, sprite, texture
atlas e animação*

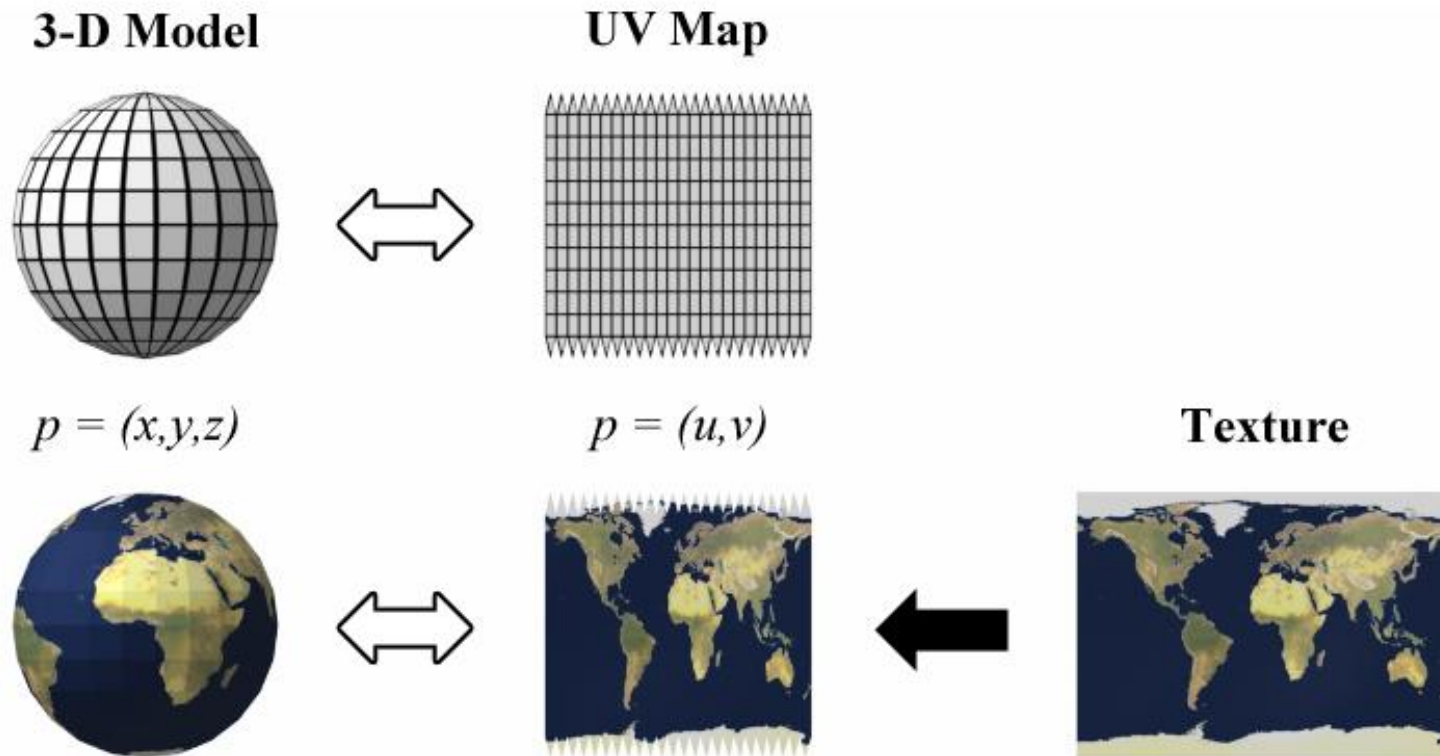


TEXTURA

- Uma imagem que foi decodificada de seu formato (ex. PNG) e enviada para a memória da GPU é **chamada de textura**.
 - Ela existe como uma matriz (sem formatos de imagem).
- Para desenhar de fato uma textura em OpenGL, **primeiro a textura é ligada (bound)**, depois existe a escrita dos dados na textura ativa.
 - O tamanho e posição da textura é **determinada pelo Viewport**.
 - Diversos jogos criam um **viewport do mesmo tamanho da resolução**, o que facilita bastante pois a geometria fica especificada em pixels e assim é fácil desenhar as texturas em tamanhos e posições apropriadas na tela.

TEXTURA

- Uma textura está sempre associada a uma geometria. Quando falamos de sprites e jogos 2D essa geometria é um retângulo.





SPRITE BATCH

- O SpriteBatch recebe a **textura e as coordenadas geométricas** que indicam onde a textura será desenhada.
 - Se ele receber uma textura **diferente da ultima textura** então ele faz o bind **dessa nova textura pra GPU**.
 - Portanto, **juntar os pedaços gráficos da sua aplicação em texturas únicas vai trazer eficiência** pois o bind das texturas não vai ser tão frequente.
- Existe um aplicativo da libgdx chamado **TexturePacker** que faz a junção automática de várias imagens em uma textura única (e ainda te dá forma de achar as regiões facilmente). Veremos mais pra frente.
- Tudo que for passado para o sprite batch deve ser desenhado dentro dos métodos **begin e end**, como previamente demonstrado.

OTIMIZAÇÃO DO SPRITE BATCH

- O SpriteBatch tem um construtor que **seta a quantidade máxima de sprites que podem ser colocados na fila** (bufferizados) antes de serem enviados para a GPU.
 - Se esse número for muito pequeno, **forçará mais chamadas e transferências de dados entre a CPU e GPU**.
 - Se esse número for muito grande, **SpriteBatch vai utilizar mais memória do que o estritamente necessário**.
 - Também é possível alterar a variável pública **maxSpritesInBatch**.
- O SpriteBatch tem um campo público chamado **renderCalls**. Depois que o método **end** for chamado, esse campo indica quantas vezes o batch de geometria foi enviado para a GPU (**dentro do último begin e end**).
 - É interessante **diminuir o valor retornado por renderCalls** para ter uma aplicação mais eficiente.



DESATIVAR O BLENDING

- Pode ser interessante também **desativar o blending na libgdx**. O blending permite a **utilização da camada alpha da sua imagem para dar transparência na hora do desenho do pixel**. Caso o blending seja desativado, a libgdx desconsiderará a transparência.
- Desativar o blending **adiciona mais eficiência na hora de desenhar**.
 - Caso não seja utilizado ou necessário, **não custa nada desabilitar**.
 - O blending vem habilitado por padrão.



DESATIVAR O BLENDING

```
Gdx.gl.glClear(GL20.GL_COLOR_BUFFER_BIT);  
batch.begin();  
batch.disableBlending();  
backgroundSprite.draw(batch);  
batch.enableBlending();  
// Outro desenho aqui.  
batch.end();
```

TEXTURE REGION

- **TextureRegion** é uma classe que fornece a possibilidade de desenhar regiões de textura ao invés da textura completa.
 - Podemos desenhar somente o sprite do passarinho, por exemplo.





TEXTURE REGION

- É possível desenhar as regiões de textura da seguinte forma:

```
private TextureRegion region;
```

```
...
```

```
texture = new Texture(Gdx.files.internal("image.png"));
```

```
region = new TextureRegion(texture, 20, 20, 50, 50);
```

```
...
```

```
batch.begin();
```

```
batch.draw(region, 10, 10);
```

```
batch.end();
```



SPRITE

- Sprite deve ser a classe mais utilizada para **desenhar objetos 2D gráficos** na tela.
- Diferente da Texture e TextureRegion, **ela possui diversos métodos particulares tais como:**
 - **setColor** : para mudar a coloração na hora do desenho.
 - **scale e setSize** : para mudar o tamanho do sprite.
 - **rotate** : para rotacionar o sprite na tela.
 - **flip** : para virar (flipar) a textura na horizontal ou na vertical.
 - **setCenter** : para setar o sprite em um centro passado como parâmetro.
 - **setAlpha** : para trocar a transparencia com que o sprite é desenhado.
 - **setPosition** : para alterar a posição de desenho do sprite.
 - **setRegion** : setar a região de textura.



SPRITE

- Para desenhar um sprite, basta fazer:

```
private Sprite sprite;
```

```
...
```

```
texture = new Texture(Gdx.files.internal("image.png"));
```

```
sprite = new Sprite(texture, 20, 20, 50, 50); //20, 20, 50, 50
```

indicam a região da textura, começa na posição 20,20 e tem largura e altura 50.

```
sprite.setPosition(10, 10);
```

```
sprite.setRotation(45);
```

```
...
```

```
batch.begin();
```

```
sprite.draw(batch);
```

```
batch.end();
```

TROCANDO A COR DO SPRITE

```
private Texture texture;
private TextureRegion region;
private Sprite sprite;
...
texture = new Texture(Gdx.files.internal("image.png"));
region = new TextureRegion(texture, 20, 20, 50, 50);
sprite = new Sprite(texture, 20, 20, 50, 50);
sprite.setPosition(100, 10);
sprite.setColor(0, 0, 1, 1);
...
batch.begin();
batch.setColor(1, 0, 0, 1);
batch.draw(texture, 10, 10);
batch.setColor(0, 1, 0, 1);
batch.draw(region, 50, 10);
sprite.draw(batch);
batch.end();
```



EXERCÍCIO

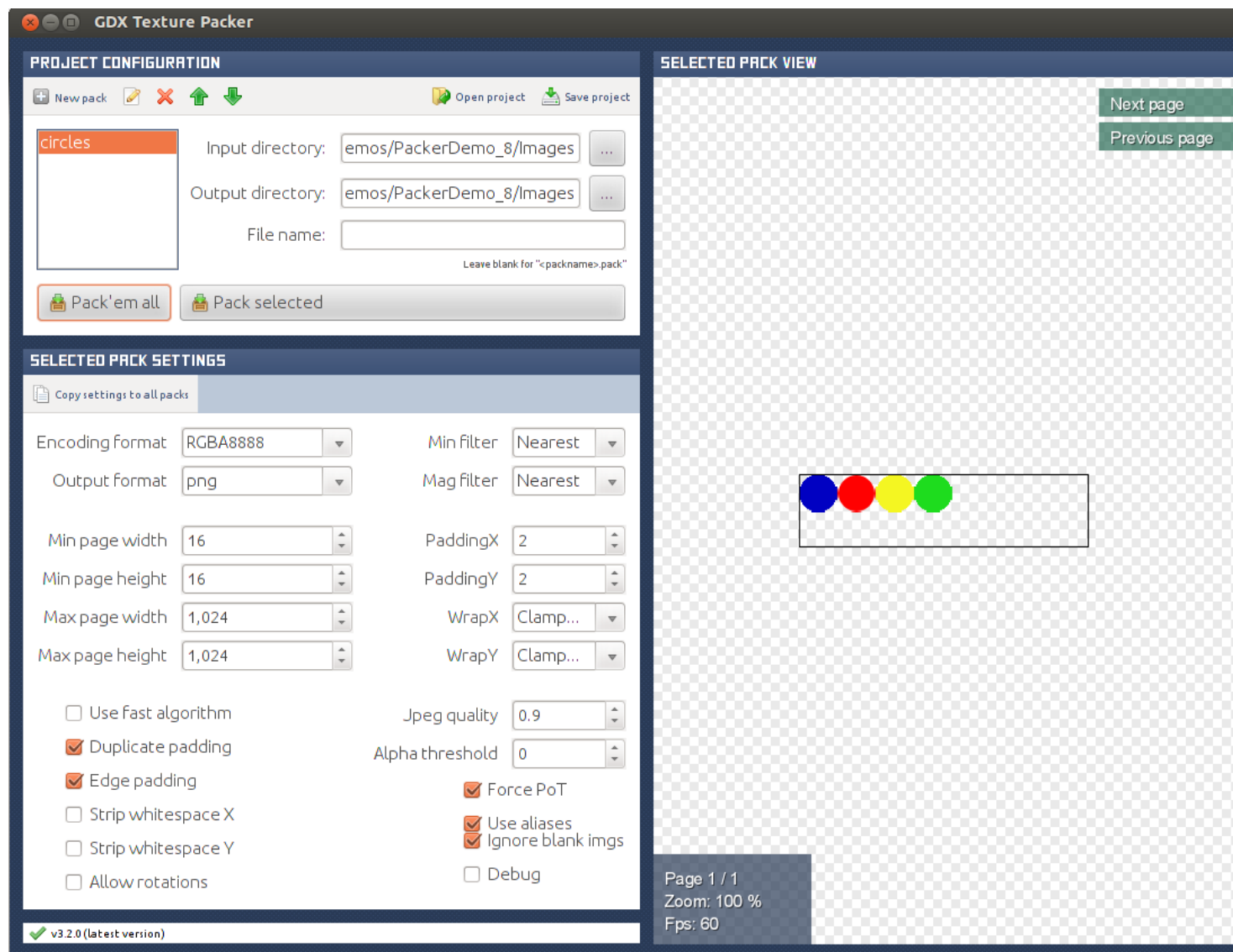
- Troque a utilização de Textures e/ou TextureRegions para **Sprites** dentro do código do seu jogo/aplicação.



TEXTURE ATLAS

- O **TexturePacker** pode ser baixado no seguinte link:
 - <https://storage.googleapis.com/google-code-archive-downloads/v2/code.google.com/libgdx-texturepacker-gui/gdx-texturepacker-3.2.0.zip>
 - <https://libgdx.badlogicgames.com/nightlies/runningables/runningable-texturepacker.jar>
- Esse programa **junta imagens menores em uma imagem maior** e adiciona um arquivo texto com a identificação das regiões da textura (que podem ser localizadas a partir do nome original das imagens).
- O **TextureAtlas** também pode e deve ser carregado a partir do AssetManager.

TEXTURE PACKER





EXERCÍCIO

- **Junte** pelo menos 2 imagens (como exercício) em um Texture Atlas.

UTILIZAÇÃO DO TEXTURE ATLAS

- Para acessar as regiões de textura de um texture atlas, você pode fazer:

```
assetManager.get(texturePath, TextureAtlas.class).findRegion(regionName);
```

- Onde o método **findRegion** encontra automaticamente (esse dado está anotado no arquivo do texture atlas) a imagem original.
- Para carregar (colocar na fila) o TextureAtlas com o AssetManager:

```
assetManager.load(path, TextureAtlas.class)
```



FORMAS (SHAPES)

- Existe uma classe chamada **ShapeRenderer** que permite o desenho de **formas simples**, como:
 - Ponto
 - Linha
 - Curva
 - Triângulo
 - Retângulo
 - Arco
 - Círculo
 - Ellipse
 - Cone
 - Polígono
- Sendo possível desenhar o conteúdo interno da forma **ou apenas a sua borda (outline)**.



FORMAS (SHAPES)

```
ShapeRenderer shape = new ShapeRenderer();  
shape.setProjectionMatrix(g.camera.combined);  
shape.begin(ShapeType.Line);  
shape.setColor(Color.RED);  
shape.rect(box.x, box.y, box.width, box.height);  
shape.end();  
  
shape.setProjectionMatrix(g.camera.combined);  
shape.begin(ShapeType.Filled);  
shape.setColor(Color.BLUE);  
shape.ellipse(box.x, box.y, box.width, box.height);  
shape.end();
```



SPRITE BATCH E SHAPE RENDERER

- Para **alternar entre o SpriteBatch e o ShapeRenderer** é preciso terminar de renderizar ambos (**método end**) e depois chamar o **método flush**, pra de fato garantir que todas as texturas ou formas tenham sido enviadas para a GPU.
- Fazer isso muitas vezes **afeta demais a performance**. O ideal é que, caso seja necessário, existe apenas uma **única alternância por frame** (por rodagem completa do método render).



EXERCÍCIO

- Com o `ShapeRenderer`, desenhe pelo menos:
 - Um `quadrado` (desenhar apenas a sua borda).
 - Uma `ellipse` completamente preenchida.
 - Um `triângulo`.

ANIMAÇÃO 2D

- A libgdx fornece uma classe para animação 2D chamada **Animation**.
- Contudo, nesse caso, eu ainda **prefiro particularmente trabalhar diretamente na classe Sprite**, assim você ainda pode utilizar os parâmetros que são inerentes à classe (**rotate**, **setposition**, **setsize**, **flip**, **etc**).
 - Para trabalhar no sprite **basta alterar a região de textura para a próxima região da animação**.





ANIMAÇÃO 2D

- Para baixar a texture com a animação de movimento (andar) do slide anterior, use o seguinte código:

HOG-KML



ANIMAÇÃO 2D

- Para alterar a região de textura do sprite basta chamar o método **setRegion**.
- A ideia é contabilizar o tempo decorrido a partir da variável delta, quando o limite para alternar entre os quadros for atingido, a **região de textura deve ser modificada**.



EXERCÍCIO

- Faça sua primeira animação 2D na libgdx utilizando a textura previamente disponibilizada ou alguma outra textura que contenha movimentação.